## Unit Testing Approach

For the Contact feature, I used JUnit tests to check both normal object creation and invalid inputs. The tests were based on the requirements for the contact ID, first name, last name, phone number, and address. I included a valid-contact test, a test for an ID that is too long, a first-name validation test, and an invalid phone number test. I also created service tests for adding a contact, preventing duplicate IDs, deleting a contact, and updating contact information. This approach was aligned with the requirements because the tests focused on the restrictions placed on each field and the required service operations. For example, ContactServiceTest.java uses assertThrows for a duplicate contact ID (lines 51–54), which directly checks the requirement that IDs must be unique.

For the Task feature, I used a similar approach with additional service-level tests. The Task tests covered valid creation, null and over-length IDs, null and over-length names, null and over-length descriptions, updating the name and description, and protecting the task ID from being changed. The TaskService tests covered adding a task, preventing duplicate IDs, deleting a task, rejecting deletion of a nonexistent task, updating the name, updating the description, and rejecting updates for nonexistent tasks. This created a direct connection between the written requirements and the behavior being tested. For example, the duplicate-ID test uses assertThrows in TaskServiceTest.java (lines 36–38).

For the Appointment feature, I tested both valid and invalid appointment data. The tests checked a valid future date, a null appointment ID, an ID longer than ten characters, a null date, a date in the past, a null description, and a description longer than fifty characters. I also tested that the description could be updated while the appointment ID and date remained unchanged. The service tests covered adding an appointment, duplicate IDs, deleting an appointment, and attempting to delete an appointment that did not exist. The invalid-date test in AppointmentTest.java (lines 68–79) is an example of using an invalid input to verify the required exception behavior.

Overall, the test approach was effective because it included both positive and negative cases rather than checking only that normal operations worked. JUnit's assertThrows is specifically intended to verify that code responds to an invalid condition with the expected exception (Bechtold et al., n.d.). The uploaded Project One archive does not contain an executed coverage report, so I cannot responsibly claim a specific numeric code-coverage percentage. Based on the tests themselves, functional coverage was broad, but it was not complete because some validation branches, such as null service objects and every possible setter failure, were not separately tested. A real coverage report would be needed to assign a defensible percentage.

## Experience Writing JUnit Tests

Writing the JUnit tests required me to turn each requirement into a specific behavior that could be checked. I tried to make the assertions match the expected result instead of simply checking

that a method ran. For example, TaskServiceTest.java updates a task and then uses assertEquals to verify the new name (lines 21–23). The Contact tests similarly use assertEquals to verify a valid contact's information. These tests are technically sound because they check the resulting state of the object instead of assuming that the method worked. The use of assertThrows for invalid inputs also makes the expected failure explicit.

The tests were efficient because each test generally created only the objects needed for its scenario and focused on one behavior. The duplicate-ID tests are a good example: they create two objects with the same identifier, add the first, and then verify that adding the second produces an exception. The deletion tests follow the same basic pattern. Keeping the setup small makes failures easier to understand and avoids combining unrelated requirements into one test. JUnit provides a standard set of assertions for this type of focused automated testing (Bechtold et al., n.d.).

**Reflection**

**Testing Techniques**

The main technique I employed was unit testing. Unit testing focuses on individual software components, which fit this project because the Contact, Task, and Appointment classes and their service classes could be tested separately. Testing the components independently made it easier to identify problems in validation and service behavior before those components would be used in a larger application. Unit testing is particularly practical for back-end services because the same tests can be run repeatedly after changes (International Software Testing Qualifications Board [ISTQB], 2024).

I also used requirements-based testing. I derived test cases from the specific requirements for each feature. When a requirement limited an ID to ten characters, I used an eleven-character ID and expected an exception. When an appointment date could not be in the past, I created a past date and expected the constructor to reject it. Requirements-based testing is useful because it connects the test cases to the behavior the software is supposed to provide. ISTQB describes black-box testing as specification-based and white-box testing as structure-based, and both can contribute to a complete testing strategy (ISTQB, 2024).

My tests also included black-box and white-box elements. The black-box side involved supplying inputs and checking the externally visible result without needing to know how the object was stored internally. The white-box side came from knowing the implementation and deliberately testing important branches, such as duplicate-ID checks and nonexistent-record checks. White-box testing is useful when the tester needs to make sure important internal decisions are exercised, while black-box testing helps confirm that the software behaves according to its requirements (ISTQB, 2024).

I did not perform integration, system, acceptance, performance, security, or usability testing in this project. Those techniques would be more appropriate for a larger application with connected

components, external systems, users, or non-functional requirements. I also did not perform a formal regression-testing cycle. Regression testing becomes especially important after software is modified because it checks that changes have not broken previously working behavior (ISTQB, 2024). In a larger project, these techniques would complement the unit tests rather than replace them.

**Mindset**

Caution was important because a passing test does not prove that the entire program is correct. I had to consider how the Contact, Task, and Appointment classes interacted with their service classes. For example, the service update methods rely on the object's setters to enforce field restrictions. That means an update can only be considered correct when both the service finds the correct object and the object accepts or rejects the new value appropriately. Software testing is intended to discover errors and increase confidence in correct behavior, so looking beyond the normal path was important (Wallace et al., 1996).

I also tried to limit bias by intentionally testing conditions that I expected to fail. It is easy for a developer to test only the normal case because the developer already knows how the code was intended to work. Using duplicate IDs, invalid lengths, null values, past dates, and nonexistent records forced me to consider how the software would behave when something went wrong. If I were responsible for testing my own production code, I would still want another developer or tester to review the tests because another person may notice assumptions that I missed.

Finally, discipline is important because cutting corners creates technical debt. Skipping a test because a method appears simple can leave a defect undiscovered and make future changes harder to trust. I plan to avoid that by writing tests alongside new functionality, keeping tests focused, rerunning the test suite after changes, and adding regression tests whenever a defect is discovered. The goal is not to create tests just to increase a coverage number. The goal is to create tests that provide useful evidence that requirements and important code paths are working as intended.

**Conclusion**

This project showed me that software testing is more than checking whether code runs. Effective testing requires connecting tests to requirements and deliberately checking both expected and unexpected behavior. The Contact, Task, and Appointment tests used JUnit assertions to verify valid operations and invalid inputs, while the service tests focused on adding, deleting, updating, duplicate records, and nonexistent records. I also learned that unit testing is only one part of a larger quality process. As a software engineering professional, I need to approach testing with caution, recognize my own bias, and remain disciplined about quality so that defects and technical debt are addressed before they become larger problems.

**References**

Bechtold, S., Brannen, S., Link, J., Merdes, M., Philipp, M., de Rancourt, J., & Stein, C. (n.d.). JUnit 5 user guide. JUnit. https://docs.junit.org/5.13.1/user-guide/index.html

International Software Testing Qualifications Board. (2024). Certified tester foundation level syllabus v4.0.1. https://www.istqb.org/wp-content/uploads/2024/11/ISTQB_CTFL_Syllabus_v4.0.1.pdf

Wallace, D. R., Watson, A. H., & McCabe, T. J. (1996). Structured testing: A testing methodology using the cyclomatic complexity metric (NIST Special Publication 500-235). National Institute of Standards and Technology. https://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication500-235.pdf